

جلسه ۴: مقدمه‌ای بر شایینی و کاربرد آن در پژوهش‌ها

بخش اول: شایینی چیست و چگونه کار می‌کند؟

۱. مقدمه

بسیاری از تحلیل‌های پژوهشی در R به صورت کدهای متوالی اجرا می‌شوند. پژوهشگر داده را وارد می‌کند، تحلیل را انجام می‌دهد و نمودار یا جدول خروجی می‌گیرد. این روش برای تحلیل گر مناسب است، اما ممکن است برای مدیر مزرعه، کارشناس آزمایشگاه یا پژوهشگری که با کدنویسی آشنایی محدودی دارد، چندان کارآمد نباشد.

کتابخانه **Shiny** چارچوبی در زبان R است که امکان ساخت برنامه‌های تعاملی تحت وب را بدون نیاز به برنامه‌نویسی مستقل وب فراهم می‌کند. با استفاده از شایینی می‌توان برنامه‌ای طراحی کرد که کاربر در آن:

- فایل داده را بارگذاری کند؛
- متغیر موردنظر را انتخاب کند؛
- گروه‌های پژوهشی را تغییر دهد؛
- بازه زمانی یا محدوده عددی را مشخص کند؛
- نمودارها را مشاهده کند؛
- خلاصه آماری دریافت کند؛
- آزمون آماری یا مدل پیش‌بینی را اجرا کند؛
- و نتایج را به صورت تعاملی بررسی کند.

۲. جایگاه شایینی در پژوهش‌های شیلات

شایینی خود یک روش آماری نیست، بلکه ابزاری برای ارائه، اجرای تعاملی و دسترسی‌پذیر کردن تحلیل‌ها است.

کاربردهای آن در پژوهش‌های شیلات و آزمون می‌تواند شامل موارد زیر باشد:

- پایش کیفیت آب در استخرهای پرورش میگو؛
- مشاهده روند رشد ماهی در طول دوره پرورش؛
- مقایسه وزن یا نرخ رشد در تیمارهای مختلف؛
- بررسی رابطه بین متغیرهای محیطی و عملکرد آبزیان؛
- نمایش نقشه یا شاخص‌های صید؛
- بررسی میزان تلفات در مخازن مختلف؛
- ساخت ابزار ساده برای تحلیل داده‌های آزمایشگاهی؛
- ارائه نتایج پژوهش به مدیران اجرایی و کارشناسان غیر آماری.

۳. معماری اصلی یک برنامه شایینی

هر برنامه ساده شایینی معمولاً از دو بخش اصلی تشکیل می‌شود:

الف) رابط کاربر یا UI

رابط کاربر مشخص می‌کند که چه چیزهایی در صفحه نمایش داده شوند:

- عنوان برنامه؛
- فهرست‌های انتخابی؛
- دکمه‌ها؛
- اسلایدرها؛
- جدول‌ها؛
- نمودارها؛
- متن‌ها و شاخص‌های عددی.

ب) منطق سرور یا Server

بخش سرور مشخص می‌کند که برنامه چگونه به ورودی‌های کاربر پاسخ دهد.

برای مثال:

- اگر کاربر گونه آبی را تغییر داد، داده‌ها فیلتر شوند؛
- اگر کاربر متغیر کیفیت آب را انتخاب کرد، نمودار جدید رسم شود؛
- اگر بازه دما تغییر کرد، خلاصه آماری دوباره محاسبه شود؛
- اگر کاربر تیمار خاصی را انتخاب کرد، جدول مربوط به همان تیمار نمایش داده شود.

ج) اجرای برنامه

تابع `shinyApp()` رابط کاربر و منطق سرور را به یک برنامه واحد متصل می‌کند.

ساختار حداقلی برنامه:

```
library(shiny)

ui <- fluidPage(
  titlePanel("Aquaculture Data Explorer")
)

server <- function(input, output, session) {
}

shinyApp(ui = ui, server = server)
```

۴. نصب و بارگذاری بسته

در صورت نصب نبودن بسته، ابتدا آن را نصب کنید:

```
install.packages("shiny")
```

سپس بسته را در هر جلسه کاری بارگذاری کنید:

```
library(shiny)
```

۵. نخستین برنامه شایینی

کد زیر یک برنامه بسیار ساده ایجاد می‌کند که در آن کاربر نام خود را وارد می‌کند و برنامه پیام مناسب را نمایش می‌دهد.

```
library(shiny)

ui <- fluidPage(
  titlePanel("Researcher Greeting"),
  sidebarLayout(
    sidebarPanel(
      textInput(
        inputId = "researcher_name",
        label = "Researcher name:",
        value = "Researcher"
      )
    ),
    mainPanel(
      textOutput("greeting_text")
    )
  )
)

server <- function(input, output, session) {
  output$greeting_text <- renderText({
    paste("Welcome,", input$researcher_name)
  })
}
```

```
}  
  
shinyApp(ui = ui, server = server)
```

۶. تشریح اجزای برنامه

تابع `fluidPage()`

این تابع ساختار کلی صفحه را ایجاد می‌کند.

تابع `titlePanel()`

برای نمایش عنوان اصلی برنامه استفاده می‌شود.

تابع `sidebarLayout()`

صفحه را به دو قسمت تقسیم می‌کند:

- برای ورودی‌ها؛ `sidebarPanel()`
- برای نمایش خروجی‌ها `mainPanel()`

تابع `textInput()`

یک کادر متنی ایجاد می‌کند. مقدار آن از طریق `input$researcher_name` در دسترس سرور قرار می‌گیرد.

تابع `textOutput()`

محل نمایش یک متن تولیدشده توسط سرور است.

تابع `renderText()`

مقدار متنی خروجی را تولید می‌کند.

۷. رابطه بین `input` و `output`

در شاینی، داده‌های واردشده توسط کاربر از طریق شیء `input` در اختیار سرور قرار می‌گیرند.
برای نمونه:

```
input$researcher_name
```

به مقدار واردشده در ورودی زیر اشاره می‌کند:

```
textInput(  
  inputId = "researcher_name",  
  label = "Researcher name:",  
  value = "Researcher"  
)
```

خروجی‌ها در شیء `output` ذخیره می‌شوند:

```
output$greeting_text
```

و در رابط کاربر با تابع متناظر نمایش داده می‌شوند:

```
textOutput("greeting_text")
```

شناسه ورودی یا خروجی باید در هر دو بخش یکسان باشد.

مثال واقعی بخش اول: نمایش اطلاعات یک مزرعه میگو

فرض کنید پژوهشگر می‌خواهد برنامه‌ای بسازد که با انتخاب نام مزرعه، اطلاعات اولیه آن را نمایش دهد.

```
library(shiny)
```

```
farm_data <- data.frame(
  farm = c("Farm_A", "Farm_B", "Farm_C"),
  area_ha = c(12, 18, 9),
  pond_count = c(8, 12, 6),
  production_ton = c(45.2, 71.5, 32.8),
  stringsAsFactors = FALSE
)

ui <- fluidPage(
  titlePanel("Shrimp Farm Information"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "selected_farm",
        label = "Select farm:",
        choices = farm_data$farm,
        selected = farm_data$farm[1]
      )
    ),
    mainPanel(
      tableOutput("farm_summary")
    )
  )
)

server <- function(input, output, session) {
  output$farm_summary <- renderTable({
    farm_data[farm_data$farm == input$selected_farm, ]
  })
}

shinyApp(ui = ui, server = server)
```

تفسیر

در این برنامه:

- نام مزرعه از طریق `selectInput()` انتخاب می‌شود.
- مقدار انتخاب‌شده در `input$selected_farm` ذخیره می‌شود.
- سرور ردیف مربوط به مزرعه را فیلتر می‌کند.
- نتیجه با `renderTable()` در صفحه نمایش داده می‌شود.

بخش دوم: ورودی‌ها، واکنش‌پذیری و محاسبات تعاملی

۱. مفهوم واکنش‌پذیری

مهم‌ترین ویژگی شاینی، واکنش‌پذیری یا `Reactivity` است.

واکنش‌پذیری یعنی خروجی برنامه به‌صورت خودکار در پاسخ به تغییر ورودی‌ها به‌روزرسانی شود.

برای مثال، اگر کاربر:

- گونه را تغییر دهد؛
- تیمار را عوض کند؛
- متغیر نمودار را انتخاب کند؛
- دامنه زمانی را تغییر دهد؛

خروجی‌های وابسته نیز به‌صورت خودکار تغییر می‌کنند.

۲. مهم‌ترین ورودی‌های شاینی

انتخاب یک گزینه با `selectInput()`

```
selectInput(  
  inputId = "species",  
  label = "Select species:",  
  choices = c("Shrimp", "Tilapia", "Sea bass"),  
  selected = "Shrimp"  
)
```

انتخاب چند گزینه با `checkboxGroupInput()`

```
checkboxGroupInput(  
  inputId = "water_variables",  
  label = "Select variables:",  
  choices = c("temperature", "salinity", "oxygen"),  
  selected = c("temperature", "oxygen")  
)
```

ورود مقدار عددی با `numericInput()`

```
numericInput(  
  inputId = "minimum_weight",  
  label = "Minimum weight:",  
  value = 10,  
  min = 0  
)
```

انتخاب بازه عددی با `sliderInput()`

```
sliderInput(  
  inputId = "temperature_range",  
  label = "Temperature range:",  
  min = 15,  
  max = 35,  
  value = c(20, 30)  
)
```

بارگذاری فایل با `fileInput()`

```
fileInput(  
  inputId = "data_file",  
  label = "Upload CSV file:",  
  accept = ".csv"  
)
```

اجرای محاسبه با `actionButton()`

```
actionButton(  
  inputId = "run_analysis",  
  label = "Run analysis"
```

```
label = "Run analysis"  
)
```

۳. خروجی‌های رایج

نوع خروجی	تابع رابط کاربر	تابع سرور
متن	textOutput()	renderText()
متن دارای قالب HTML	htmlOutput()	renderUI()
جدول ساده	tableOutput()	renderTable()
جدول تعاملی	DTOutput()	renderDT()
نمودار	plotOutput()	renderPlot()
مقدار عددی یا شاخص	verbatimTextOutput()	renderPrint()

برای استفاده از جدول تعاملی:

```
install.packages("DT")  
library(DT)
```

۴. ساخت یک داده واکنشی با reactive()

گاهی چند خروجی باید از یک داده فیلترشده مشترک استفاده کنند. در این حالت، بهتر است داده فیلترشده در یک عبارت واکنشی ذخیره شود.

```
filtered_data <- reactive({  
  dataset[dataset$group == input$selected_group, ]  
})
```

برای استفاده از نتیجه، باید آن را مانند تابع فراخوانی کرد:

```
filtered_data()
```

۵. نمونه برنامه برای فیلتر داده‌های کیفیت آب

```
library(shiny)  
library(ggplot2)  
  
water_data <- data.frame(  
  pond = rep(paste0("P", 1:6), each = 4),  
  week = rep(1:4, times = 6),  
  temperature = c(  
    27.1, 27.8, 28.4, 29.0,  
    26.5, 27.0, 27.6, 28.2,  
    28.0, 28.6, 29.1, 29.7,  
    25.8, 26.4, 26.9, 27.5,  
    27.3, 27.9, 28.5, 28.8,  
    26.9, 27.4, 28.0, 28.6  
  ),  
  salinity = c(  
    18.2, 18.5, 18.1, 17.9,  
    20.1, 20.4, 20.0, 19.8,  
    16.8, 17.1, 17.3, 17.0,  
    22.0, 21.7, 21.5, 21.2,  
    19.0, 19.2, 19.5, 19.3,  
    17.8, 18.0, 18.4, 18.2  
  ),  
)
```

```

    dissolved_oxygen = c(
      5.8, 5.5, 5.2, 4.9,
      6.2, 5.9, 5.7, 5.4,
      5.5, 5.2, 4.8, 4.6,
      6.4, 6.1, 5.8, 5.6,
      5.9, 5.7, 5.4, 5.1,
      6.0, 5.8, 5.5, 5.3
    )
  )
)

ui <- fluidPage(
  titlePanel("Water Quality Explorer"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "selected_pond",
        label = "Select pond:",
        choices = unique(water_data$pond),
        selected = unique(water_data$pond)[1]
      ),
      selectInput(
        inputId = "selected_variable",
        label = "Select variable:",
        choices = c(
          "Temperature" = "temperature",
          "Salinity" = "salinity",
          "Dissolved oxygen" = "dissolved_oxygen"
        ),
        selected = "temperature"
      )
    ),
    mainPanel(
      plotOutput("water_plot"),
      tableOutput("water_summary")
    )
  )
)

server <- function(input, output, session) {
  selected_data <- reactive({
    water_data[water_data$pond == input$selected_pond, ]
  })

  output$water_plot <- renderPlot({
    data_to_plot <- selected_data()

    ggplot(
      data_to_plot,
      aes(x = week, y = .data[[input$selected_variable]])
    ) +
    geom_line(linewidth = 1, color = "steelblue") +
    geom_point(size = 3, color = "darkred") +
    labs(
      x = "Week",
      y = input$selected_variable,
      title = paste("Water quality in", input$selected_pond)
    ) +
    theme_minimal()
  })

  output$water_summary <- renderTable({

```

```
data_to_summarize <- selected_data()
variable_values <- data_to_summarize[[input$selected_variable]]

data.frame(
  minimum = min(variable_values),
  mean = mean(variable_values),
  maximum = max(variable_values),
  standard_deviation = sd(variable_values)
)
})
}

shinyApp(ui = ui, server = server)
```

۶. نکته مهم درباره `data[...]`

در کد زیر:

```
aes(x = week, y = .data[[input$selected_variable]])
```

نام متغیر نمودار از ورودی کاربر دریافت می‌شود.

چون نام متغیر به صورت پویا تغییر می‌کند، نمی‌توان آن را به صورت ثابت نوشت:

```
aes(x = week, y = temperature)
```

استفاده از `data[...]` به برنامه اجازه می‌دهد متغیر انتخاب‌شده را به صورت پویا در نمودار قرار دهد.

۷. استفاده از `req()`

اگر اجرای یک خروجی به وجود یک ورودی یا داده خاص وابسته باشد، می‌توان از `req()` استفاده کرد.

```
output$water_plot <- renderPlot({
  req(input$selected_pond)
  req(input$selected_variable)

  data_to_plot <- selected_data()

  plot(
    data_to_plot$week,
    data_to_plot[[input$selected_variable]]
  )
})
```

تابع `req()` از اجرای کد در شرایط ناقص جلوگیری می‌کند.

۸. کنترل اجرای تحلیل با `observeEvent()`

اگر بخواهیم تحلیل فقط پس از فشردن یک دکمه اجرا شود، از `observeEvent()` استفاده می‌کنیم.

```
analysis_result <- eventReactive(input$run_analysis, {
  mean(water_data$dissolved_oxygen)
})

output$mean_oxygen <- renderText({
  req(analysis_result())
  round(analysis_result(), 2)
})
```


- `run_analysis` دکمه اجرای تحلیل است.
- `eventReactive()` فقط پس از فشردن دکمه اجرا می‌شود.

- نتیجه در خروجی نمایش داده می‌شود.

مثال واقعی بخش دوم: بررسی اثر محدوده اکسیژن محلول بر مخازن پرورش

فرض کنید پژوهشگر می‌خواهد فقط مخازنی را بررسی کند که میانگین اکسیژن محلول آن‌ها در یک محدوده مشخص قرار دارد.

```
library(shiny)
library(dplyr)
library(ggplot2)

pond_data <- data.frame(
  pond = paste0("P", 1:10),
  treatment = rep(c("Control", "Supplement"), each = 5),
  mean_oxygen = c(4.8, 5.1, 5.4, 5.7, 6.0, 5.0, 5.6, 6.1, 6.4, 6.8),
  survival = c(68, 72, 75, 78, 80, 74, 79, 83, 86, 88)
)

ui <- fluidPage(
  titlePanel("Pond Survival Analysis"),
  sidebarLayout(
    sidebarPanel(
      sliderInput(
        inputId = "oxygen_range",
        label = "Mean dissolved oxygen range:",
        min = min(pond_data$mean_oxygen),
        max = max(pond_data$mean_oxygen),
        value = range(pond_data$mean_oxygen),
        step = 0.1
      )
    ),
    mainPanel(
      plotOutput("survival_plot"),
      tableOutput("selected_ponds")
    )
  )
)

server <- function(input, output, session) {
  filtered_ponds <- reactive({
    pond_data %>%
      filter(
        mean_oxygen >= input$oxygen_range[1],
        mean_oxygen <= input$oxygen_range[2]
      )
  })

  output$survival_plot <- renderPlot({
    ggplot(
      filtered_ponds(),
      aes(x = mean_oxygen, y = survival, color = treatment)
    ) +
    geom_point(size = 3) +
    geom_smooth(method = "lm", se = FALSE) +
    labs(
      x = "Mean dissolved oxygen",
      y = "Survival percentage",
      color = "Treatment"
    )
  })
}
```

```
) +
theme_minimal()
})

output$selected_ponds <- renderTable({
  filtered_ponds()
})
}

shinyApp(ui = ui, server = server)
```

تفسیر

این برنامه به پژوهشگر اجازه می‌دهد:

- محدوده اکسیژن محلول را تغییر دهد؛
- مخازن موجود در محدوده انتخاب شده را مشاهده کند؛
- رابطه اکسیژن محلول و زنده‌مانی را بررسی کند؛
- تفاوت الگوی دو تیمار را به صورت بصری مقایسه کند.

این نمودار به تنهایی اثبات‌کننده رابطه علی نیست. برای نتیجه‌گیری علمی، باید طراحی مطالعه، تعداد تکرارها، اثرات مزرعه و سایر عوامل مؤثر نیز بررسی شوند.

بخش سوم: ساخت داشبورد ساده برای تحلیل داده‌های پژوهشی

۱. داشبورد چیست؟

داشبورد یک محیط تعاملی برای نمایش هم‌زمان چند نوع اطلاعات است. در پژوهش‌های شیلات، داشبورد می‌تواند شامل موارد زیر باشد:

- شاخص‌های عددی؛
- جدول داده‌ها؛
- نمودار روند؛
- نمودار مقایسه گروه‌ها؛
- فیلترهای پژوهشی؛
- خلاصه آماری؛
- خروجی مدل یا آزمون.

تفاوت داشبورد با یک نمودار ساده در این است که داشبورد معمولاً چند خروجی و چند کنترل تعاملی را در کنار یکدیگر قرار می‌دهد.

۲. ساختار صفحه با `navbarPage()`

برای ایجاد چند زبانه می‌توان از `navbarPage()` استفاده کرد.

```
ui <- navbarPage(
  title = "Aquaculture Dashboard",
  tabPanel(
    "Overview",
    ...
  ),
  tabPanel(
    "Growth",
    ...
  ),
  tabPanel(
    "Water quality",
    ...
  )
)
```

هر `tabPanel()` یک بخش مستقل از برنامه ایجاد می‌کند.

۳. نمایش شاخص‌های کلیدی با `valueBox()`

تابع `valueBox()` در بسته `shinydashboard` برای نمایش شاخص‌های عددی کاربرد دارد.

```
install.packages("shinydashboard")
```

```
library(shinydashboard)
```

```
valueBox(  
  value = "82.4%",  
  subtitle = "Mean survival",  
  icon = icon("chart-line"),  
  color = "green"  
)
```

برای استفاده از این ساختار معمولاً از `dashboardSidebar` ، `dashboardHeader()` ، `dashboardPage()` و `dashboardBody()` استفاده می‌شود.

۴. نمونه ساختار داشبورد

```
library(shiny)  
library(shinydashboard)  
  
ui <- dashboardPage(  
  dashboardHeader(title = "Aquaculture Dashboard"),  
  dashboardSidebar(  
    sidebarMenu(  
      menuItem("Overview", tabName = "overview", icon = icon("dashboard")),  
      menuItem("Growth", tabName = "growth", icon = icon("fish")),  
      menuItem("Water quality", tabName = "water", icon = icon("water"))  
    )  
  ),  
  dashboardBody(  
    tabItems(  
      tabItem(  
        tabName = "overview",  
        h2("Overview")  
      ),  
      tabItem(  
        tabName = "growth",  
        h2("Growth analysis")  
      ),  
      tabItem(  
        tabName = "water",  
        h2("Water quality analysis")  
      )  
    )  
  )  
)  
  
server <- function(input, output, session) {  
}  
  
shinyApp(ui = ui, server = server)
```

۵. نمایش نمودار تعاملی با `plotly`

برای تعاملی کردن نمودارهای `ggplot2` می‌توان از بسته `plotly` استفاده کرد.

```
install.packages("plotly")
```

```
library(plotly)

interactive_plot <- ggplot(
  shrimp_data,
  aes(x = week, y = weight, color = treatment)
) +
  geom_line() +
  geom_point()

ggplotly(interactive_plot)
```

در شاینی:

```
plotlyOutput("growth_plot")
```

و در بخش سرور:

```
output$growth_plot <- renderPlotly({
  ggplotly(
    ggplot(
      filtered_data(),
      aes(x = week, y = weight, color = treatment)
    ) +
    geom_line() +
    geom_point()
  )
})
```

۶. نمایش جدول تعاملی با DT

برای نمایش داده‌ها در قالب جدول قابل جست‌وجو و مرتب‌سازی:

```
install.packages("DT")
```

```
library(DT)

DTOutput("data_table")
```

در سرور:

```
output$data_table <- renderDT({
  datatable(
    filtered_data(),
    options = list(
      pageLength = 10,
      scrollX = TRUE
    )
  )
})
```

۷. بارگذاری داده پژوهشی

```
ui <- fluidPage(  
  fileInput(  
    inputId = "uploaded_file",  
    label = "Upload a CSV file:",  
    accept = ".csv"  
  ),  
  tableOutput("uploaded_table")  
)  
  
server <- function(input, output, session) {  
  uploaded_data <- reactive({  
    req(input$uploaded_file)  
  
    read.csv(  
      input$uploaded_file$datapath,  
      stringsAsFactors = FALSE  
    )  
  })  
  
  output$uploaded_table <- renderTable({  
    head(uploaded_data())  
  })  
}
```

۸. کنترل خطا در بارگذاری داده

در یک برنامه واقعی، باید بررسی شود که:

- فایل انتخاب شده باشد؛
- فرمت فایل درست باشد؛
- ستون‌های موردنیاز وجود داشته باشند؛
- تعداد داده‌ها کافی باشد؛
- متغیرهای عددی واقعاً به صورت عددی خوانده شده باشند.

نمونه بررسی ستون‌ها:

```
required_columns <- c(  
  "pond",  
  "week",  
  "treatment",  
  "weight",  
  "survival"  
)  
  
missing_columns <- setdiff(  
  required_columns,  
  names(uploaded_data())  
)  
  
validate(  
  need(  
    length(missing_columns) == 0,  
    "Required columns are missing from the uploaded file."  
  )  
)
```

```
output$summary_table <- renderTable({
  selected_data() %>%
    summarise(
      n = n(),
      mean_weight = mean(weight, na.rm = TRUE),
      sd_weight = sd(weight, na.rm = TRUE),
      min_weight = min(weight, na.rm = TRUE),
      max_weight = max(weight, na.rm = TRUE)
    )
})
```

۱۰. افزودن تحلیل آماری به داشبورد

شاینی می‌تواند یک آزمون یا مدل را بر اساس انتخاب‌های کاربر اجرا کند. برای مثال، مقایسه وزن نهایی بین دو تیمار:

```
output$test_result <- renderPrint({
  selected_data <- growth_data[
    growth_data$treatment %in% input$selected_treatments,
  ]

  t.test(
    final_weight ~ treatment,
    data = selected_data
  )
})
```

با وجود این، در یک برنامه پژوهشی نباید اجرای آزمون آماری به صورت کاملاً بدون کنترل انجام شود. باید:

- حداقل تعداد نمونه بررسی شود؛
- تعداد گروه‌ها کنترل شود؛
- داده‌های گمشده مدیریت شوند؛
- مفروضات آزمون بررسی شوند؛
- هشدارها و خطاها به کاربر نمایش داده شوند.

۱۱. یک داشبورد کامل نمونه

کد زیر یک داشبورد ساده برای بررسی رشد و زنده‌مانی میگو ایجاد می‌کند.

```
library(shiny)
library(shinydashboard)
library(dplyr)
library(ggplot2)
library(plotly)
library(DT)

set.seed(123)

shrimp_data <- expand_grid(
  pond = paste0("P", 1:8),
  week = 1:6
)

shrimp_data$treatment <- rep(
  c("Control", "Supplement"),
  length.out = nrow(shrimp_data)
```

```

)

shrimp_data$weight <- round(
  2 + shrimp_data$week * 1.8 +
  ifelse(shrimp_data$treatment == "Supplement", 1.2, 0) +
  rnorm(nrow(shrimp_data), 0, 0.5),
  2
)

shrimp_data$survival <- round(
  70 + shrimp_data$week * 1.5 +
  ifelse(shrimp_data$treatment == "Supplement", 4, 0) +
  rnorm(nrow(shrimp_data), 0, 2),
  1
)

ui <- dashboardPage(
  dashboardHeader(title = "Shrimp Production Dashboard"),
  dashboardSidebar(
    sidebarMenu(
      menuItem(
        "Overview",
        tabName = "overview",
        icon = icon("dashboard")
      ),
      menuItem(
        "Growth",
        tabName = "growth",
        icon = icon("chart-line")
      ),
      menuItem(
        "Data",
        tabName = "data",
        icon = icon("table")
      )
    )
  ),
  dashboardBody(
    tabItems(
      tabItem(
        tabName = "overview",
        fluidRow(
          valueBoxOutput("mean_weight_box"),
          valueBoxOutput("mean_survival_box")
        ),
        fluidRow(
          box(
            width = 12,
            title = "Survival trend",
            plotlyOutput("survival_plot")
          )
        )
      ),
      tabItem(
        tabName = "growth",
        fluidRow(
          box(
            width = 4,
            selectInput(
              inputId = "selected_treatment",
              label = "Select treatment:",

```



```

    color = treatment,
    group = interaction(pond, treatment),
    text = paste(
      "Pond:", pond,
      "<br>Week:", week,
      "<br>Weight:", weight
    )
  )
} +
geom_line(alpha = 0.6) +
geom_point() +
labs(
  x = "Week",
  y = "Weight",
  color = "Treatment"
) +
theme_minimal()

ggplotly(growth_plot, tooltip = "text")
})

output$survival_plot <- renderPlotly({
  survival_summary <- filtered_data() %>%
    group_by(week, treatment) %>%
    summarise(
      mean_survival = mean(survival),
      .groups = "drop"
    )

  survival_plot <- ggplot(
    survival_summary,
    aes(
      x = week,
      y = mean_survival,
      color = treatment,
      group = treatment
    )
  ) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    x = "Week",
    y = "Mean survival",
    color = "Treatment"
  ) +
  theme_minimal()

  ggplotly(survival_plot)
})

output$data_table <- renderDT({
  datatable(
    filtered_data(),
    options = list(
      pageLength = 10,
      scrollX = TRUE
    )
  )
})
})
}

```

۱۲. تفسیر داشبورد

این داشبورد چند قابلیت دارد:

- فیلتر کردن داده‌ها بر اساس تیمار؛
- محاسبه میانگین وزن؛
- محاسبه میانگین زنده‌مانی؛
- نمایش روند رشد؛
- نمایش روند زنده‌مانی؛
- نمایش داده‌ها در جدول تعاملی؛
- مشاهده اطلاعات نقاط نمودار با قرار دادن نشانگر روی آن‌ها.

با تغییر تیمار، خروجی‌های وابسته نیز تغییر می‌کنند. این همان مفهوم واکنش‌پذیری است.

۱۳. ملاحظات علمی در طراحی داشبورد

داشبورد تعاملی نباید باعث شود تحلیل آماری بدون توجه به طراحی پژوهش انجام شود.

در پروژه‌های شیلاتی باید به موارد زیر توجه کرد:

- واحد آزمایشی واقعی چیست؟
- آیا نمونه‌ها مستقل هستند؟
- آیا مخزن، مزرعه یا فرد آبری واحد تحلیل است؟
- آیا داده‌ها تکرارهای فنی یا زیستی دارند؟
- آیا اندازه نمونه برای مقایسه کافی است؟
- آیا متغیرها مقیاس یکسان دارند؟
- آیا داده‌های گمشده به‌درستی مدیریت شده‌اند؟
- آیا نمودارهای تعاملی ممکن است باعث برداشت نادرست شوند؟
- آیا نمایش یک رابطه به‌عنوان رابطه علی از نظر علمی مجاز است؟

۱۴. محدودیت‌های شاینی

برنامه‌های شاینی ابزارهای قدرتمندی هستند، اما محدودیت دارند:

- کیفیت داشبورد به کیفیت داده و تحلیل وابسته است.
- واکنش‌پذیری زیاد ممکن است برنامه را کند کند.
- تحلیل‌های سنگین ممکن است نیازمند بهینه‌سازی باشند.
- داده‌های حساس پژوهشی نباید بدون کنترل در سامانه‌های عمومی بارگذاری شوند.
- خروجی داشبورد جایگزین گزارش کامل روش‌شناسی پژوهش نیست.
- کاربر ممکن است با تغییر فیلترها به مقایسه‌های متعدد و کنترل‌نشده برسد.
- داشبورد باید همراه با مستندسازی روش تحلیل ارائه شود.

مثال واقعی بخش سوم: داشبورد پایش رشد و کیفیت آب در مزارع میگو

فرض کنید یک گروه پژوهشی داده‌های زیر را از چند مزرعه میگو جمع‌آوری کرده است:

- شناسه مزرعه؛
- شناسه مخزن؛
- هفته نمونه‌برداری؛
- دمای آب؛
- شوری؛
- اکسیژن محلول؛

- pH؛

- وزن میانگین میگو؛
- درصد زنده‌مانی؛
- نوع خوراک.

هدف طراحی برنامه‌ای است که کاربر بتواند:

۱. یک مزرعه یا مخزن را انتخاب کند؛
۲. روند تغییر کیفیت آب را مشاهده کند؛
۳. روند رشد میگو را بررسی کند؛
۴. میانگین زنده‌مانی را با سایر مخازن مقایسه کند؛
۵. داده‌ها را در جدول جست‌وجو کند؛
۶. فایل داده پالایش‌شده را دریافت کند.

یک ساختار داده پیشنهادی:

```
farm_data <- data.frame(  
  farm_id = c("F1", "F1", "F1", "F2", "F2"),  
  pond_id = c("P1", "P2", "P3", "P1", "P2"),  
  week = c(1, 1, 1, 1, 1),  
  temperature = c(28.1, 28.4, 27.9, 29.0, 28.7),  
  salinity = c(18.2, 19.0, 17.8, 21.1, 20.5),  
  dissolved_oxygen = c(5.8, 5.4, 6.1, 4.9, 5.2),  
  ph = c(7.8, 7.7, 8.0, 7.6, 7.8),  
  mean_weight = c(2.4, 2.6, 2.3, 2.5, 2.7),  
  survival = c(82.1, 79.4, 85.0, 76.2, 78.8),  
  feed_type = c("Feed_A", "Feed_A", "Feed_B", "Feed_A", "Feed_B")  
)
```

پرسش‌های پژوهشی قابل بررسی در داشبورد

- کدام مزرعه میانگین زنده‌مانی بیشتری دارد؟
- آیا کاهش اکسیژن محلول با کاهش زنده‌مانی همراه است؟
- روند رشد میگو در کدام نوع خوراک بهتر است؟
- آیا تفاوت میان مخازن یک مزرعه بیشتر از تفاوت میان مزارع است؟
- آیا مخازنی با شوری مشابه، الگوی رشد مشابهی دارند؟
- در چه هفته‌ای بیشترین کاهش زنده‌مانی رخ داده است؟

ملاحظات تفسیری

نتایج داشبورد باید به‌عنوان ابزار اکتشافی و تصمیم‌یار تفسیر شوند. برای انتشار مقاله، لازم است تحلیل نهایی با روش‌های آماری مناسب، تعریف دقیق واحد آزمایشی، بررسی مفروضات و گزارش کامل عدم قطعیت انجام شود.

پروژه پایانی جلسه

طراحی داشبورد تعاملی برای پایش رشد میگو

یک برنامه شاینی برای تحلیل داده‌های رشد و کیفیت آب در چند مخزن پرورش میگو طراحی کنید.

داده پیشنهادی

برای هر مشاهده، حداقل متغیرهای زیر را در نظر بگیرید:

- farm_id
- pond_id
- week
- treatment
- temperature
- salinity
- dissolved_oxygen
- ph
- mean_weight

- survival
- feed_type

قابلیت‌های الزامی برنامه

۱. بارگذاری فایل داده با فرمت CSV؛
۲. نمایش پیش‌نمایش داده‌ها؛
۳. انتخاب مزرعه؛
۴. انتخاب مخزن؛
۵. انتخاب تیمار یا نوع خوراک؛
۶. نمایش میانگین وزن؛
۷. نمایش میانگین زنده‌مانی؛
۸. نمایش روند وزن میانگین در طول زمان؛
۹. نمایش روند اکسیژن محلول و دما؛
۱۰. نمایش جدول داده‌های فیلترشده؛
۱۱. نمایش خلاصه آماری متغیر انتخاب‌شده؛
۱۲. امکان دریافت داده فیلترشده.

قابلیت‌های پیشنهادی

- افزودن نمودار پراکنش اکسیژن محلول و زنده‌مانی؛
- افزودن خط رگرسیون به نمودار پراکنش؛
- افزودن هشدار برای اکسیژن محلول کمتر از آستانه تعیین‌شده؛
- افزودن انتخاب بازه زمانی؛
- افزودن مقایسه میانگین تیمارها؛
- افزودن خروجی قابل دریافت از نمودار یا جدول؛
- استفاده از `plotly` برای نمودارهای تعاملی؛
- استفاده از `DT` برای جدول قابل جست‌وجو؛
- بررسی وجود ستون‌های ضروری پیش از اجرای برنامه.

راهنمای انجام پروژه

- برنامه را در یک فایل با نام `app.R` ایجاد کنید.
- ابتدا رابط کاربر و منطق سرور را به‌صورت جداگانه طراحی کنید.
- داده‌ها را در یک شیء واکنشی قرار دهید.
- فیلترهای مختلف را در یک یا چند عبارت `reactive()` پیاده‌سازی کنید.
- برای خروجی‌های گرافیکی از `renderPlot()` یا `renderPlotly()` استفاده کنید.
- برای خروجی‌های عددی از `renderText()` یا `renderValueBox()` استفاده کنید.
- قبل از اجرای تحلیل‌ها، داده‌های گمشده و تعداد مشاهدات را بررسی کنید.
- برای هر نمودار، عنوان، نام محورها و واحد اندازه‌گیری را مشخص کنید.
- در گزارش همراه پروژه، توضیح دهید که داشبورد برای چه نوع تصمیم یا پرسش پژوهشی طراحی شده است.
- از نتیجه‌گیری علّی بر اساس نمودارهای توصیفی خودداری کنید.
- راهنمای استفاده از برنامه و تعریف متغیرها را در قالب یک فایل `README.md` بنویسید.